

Transactional Email Module Specification

Overview

Create a transactional email module for the Payload CMS application that automatically sends templated emails when CRM-related events occur.

Current system behavior:

- Successful Stripe events create CRM records.
- Customer email addresses are collected through Stripe and intake flows.
- CRM currently stores customer/contact information but does not trigger automated transactional communication.

The new module will introduce an email delivery layer that sends branded, templated emails based on CRM events.

Goals

- Send immediate transactional emails after key customer actions.
- Maintain reusable email templates.
- Separate email delivery logic from Stripe/webhook handlers.
- Allow future expansion into additional lifecycle emails.
- Track email delivery status.

Recommended Stack

Email Provider

Primary recommendation: Resend

Reasoning:

- Developer-focused API.
- Excellent React Email integration.
- Modern transactional email workflow.
- Strong Next.js compatibility.
- Simple template development.
- Good deliverability tooling.

Existing alternative:

Brevo

Pros:

- Existing account and email marketing history.
- Existing contacts/lists.
- Marketing campaign capabilities.

Cons:

- Less developer-oriented transactional workflow.
- Template development experience is weaker compared with React Email.
- Transactional and marketing concerns become mixed.

Recommendation:

Use Resend for transactional emails.

Continue using Brevo for:

- newsletters
- campaigns
- broadcasts
- eblasts

Architecture

```
Stripe Event
  |
  v
Webhook Handler
  |
  v
CRM Record Creation
  |
  v
Event Dispatcher
  |
  v
Email Template Resolver
  |
  v
Resend API
  |
  v
Customer Email
```

Supported Email Triggers

Event 1: Successful Stripe Payment

Trigger:

Stripe payment succeeds.

Examples:

- portrait commission purchased
- event drawing purchased

- other paid product/service

Actions:

1. Create/update CRM record.
2. Dispatch `customer.purchase.completed`.
3. Send confirmation email.

Template:

`purchase-confirmation`

Variables:

- customer name
 - product/service name
 - order amount
 - purchase date
 - next steps
 - support contact
-

Event 2: Stripe Checkout Completion

Trigger:

Stripe checkout session completed.

Purpose:

Capture customers who complete checkout but require additional workflow.

Actions:

1. Verify customer data.
2. Update CRM.
3. Dispatch `checkout.completed`.
4. Send follow-up email.

Template:

`checkout-thank-you`

Variables:

- customer name
 - order information
 - intake instructions
 - upload link
-

Event 3: CRM Intake Form Submission

Trigger:

Customer submits portrait intake form.

Actions:

1. Create/update CRM record.
2. Dispatch `intake.submitted`.
3. Send confirmation email.

Template:

`intake-confirmation`

Variables:

- customer name
- pet name
- submitted images
- expected timeline
- next steps

Email Module Structure

Suggested Payload structure:

```
src/  
  modules/  
    email/  
      index.ts  
      email.service.ts  
      email.events.ts  
      email.templates.ts  
      email.types.ts  
      providers/  
        resend.ts  
      templates/  
        PurchaseConfirmation.tsx  
        IntakeConfirmation.tsx  
        CheckoutThankYou.tsx
```

Email Service

Responsibilities:

- Accept email events.
- Resolve templates.
- Inject data.
- Send through provider.
- Log results.

Example interface:

```
sendEmail({  
  event: "customer.purchase.completed",  
  recipient,  
  data  
})
```

Event System

Create a lightweight internal event dispatcher.

Example:

```
emitEvent(  
  "customer.purchase.completed",  
  {  
    customerId,  
    email,  
    orderId  
  }  
)
```

Benefits:

- Stripe code stays clean.
- Future automation becomes easier.
- Multiple actions can respond to events.

Future listeners:

- email notification
- CRM tagging
- analytics tracking
- Slack notifications

React Email Templates

Use React components for templates.

Benefits:

- Component reuse.
- Easier styling.
- Local preview.
- Better developer workflow.

Template requirements:

- Mobile responsive.
- Plain-text fallback.
- Brand styling.
- Minimal HTML complexity.

CRM Integration

Add fields:

Email Status:

- queued
- sent
- delivered
- failed

Email History:

- event name
- template
- timestamp
- provider message ID
- failure reason

Example:

```
Customer
|
+-- Email Events
    |
    +-- purchase-confirmation
    +-- intake-confirmation
```

Environment Variables

Required:

```
RESEND_API_KEY=
EMAIL_FROM=
EMAIL_REPLY_TO=
```

Admin Features

Phase 1:

- automatic sending only
- logging

Phase 2:

- resend failed emails
- preview templates
- manual send

Phase 3:

- CRM email timeline
- customer communication history
- template editor

Error Handling

Failures should not block CRM creation.

Example:

Stripe payment succeeds:

1. Create CRM record.
2. Attempt email.
3. If email fails:
 - log failure
 - retry
 - notify admin

Email failure should be recoverable.

Security

Requirements:

- Validate webhook signatures.
- Do not expose Resend keys client-side.
- Sanitize template variables.
- Rate limit manual sends.

Implementation Order

1. Add Resend provider.
2. Add React Email template system.
3. Add email service abstraction.
4. Add event dispatcher.
5. Connect Stripe purchase event.
6. Connect checkout event.
7. Connect intake form event.
8. Add CRM email history.
9. Add admin controls.

Future Expansion

Possible future emails:

- portrait completion notification
- shipping notification
- review request
- repeat customer campaigns
- abandoned checkout follow-up
- event reminders

Final Recommendation

Implement Resend + React Email as the transactional email layer.

Keep Brevo as the marketing/email blast platform.

The architecture should treat email as an independent module driven by CRM events rather than embedding email logic inside Stripe handlers.